

# A Feasible Schema Design Strategy for Amazon DynamoDB: A Nested Normal Form Approach

Wai Yin Mok

Department of Information Systems, The University of Alabama in Huntsville, Huntsville, USA  
(mokw@uah.edu)

**Abstract** – Amazon DynamoDB is a next-gen NoSQL (Not only SQL), key-value and document database that delivers single-digit millisecond performance at any scale. As more complex web-based applications adopt DynamoDB, it is imperative to develop sound design strategies for DynamoDB schemas. Large-scale web-based applications often exhibit conflicting requirements. Hence, the research problem is to design the DynamoDB schemas of a given application with conflicting requirements such that the database must satisfy certain predetermined performance goals. In the era of relational databases, normal forms were developed to guide the schema design process. Past knowledge and experiences, we believe, are also applicable to DynamoDB schema design. Specifically, based upon Nested Normal Form and XML databases, this paper demonstrates the feasibility of a design strategy on DynamoDB schemas, particularly with the access patterns of the data in mind. Simulation further substantiates the feasibility of our approach.

**Keywords** – Amazon DynamoDB Schema Design, Nested Normal Form, XML Databases

## I. INTRODUCTION

As large-scale web-based applications demand speed, flexibility and scalability, NoSQL database systems, a next-gen database systems, are being developed. Many commercial NoSQL database systems are now available. Notable examples include MongoDB [10], Cassandra [5], Redis [12] and Amazon DynamoDB [1] and others. The underlying data models of NoSQL databases are quite different from the tabular data model of the traditional relational databases. Some prevalent data models for NoSQL databases are the document model, the graph model and the key-value model [11]. Amazon DynamoDB adopts the key-value model and the document model, and many top-level entities of a database are modeled as nodes and their relationships as edges as in graphs [1].

The results of system analysis, an indispensable activity of any large-scale system development, include many different types of diagrams focusing on various aspects of the system under development. UML diagrams [4], promoted by robust modeling tools like Visual Paradigm [13] and others, have become a software industry standard. The proposed DynamoDB schema design strategy also begins with a graphical diagram, called a conceptual-model hypergraph, which is in a way similar to but simpler than UML domain model class diagrams. The purpose of such a hypergraph, like a UML domain model class diagram, is to capture the static, rather than the dynamic, aspects of a system of interest.

Nested Normal Form [9] was originally designed for nested relational databases, which allow tables to be nested within other tables, resulting in embedded tables and thus hierarchical structures. Since XML databases [3] also allow hierarchical structures, it is natural to extend Nested Normal Form to XML databases. A critical contribution of Nested Normal Form is that XML databases that conform to Nested Normal Form are guaranteed to be free of unwanted redundant data. In [8], we devised algorithms that generate Nested Normal Form XML databases. We shall shortly show that the algorithms in [8], with some modifications, are also applicable to DynamoDB as well.

We believe conceptual-model hypergraphs and the XML database design algorithms form the conceptual framework in which the solution for the research problem stated in the abstract can be formulated. In contrast, in one single page the documentation of Amazon DynamoDB states some NoSQL design principles for DynamoDB schemas. Although the suggested design principles are sound, they are so vague to formulate a formal DynamoDB design strategy.

To convey the central idea of the proposed design strategy, the rigorous mathematics underlying [7,8,9] is avoided. Instead, this paper adopts a high-level, step-by-step presentation. To proceed, Section 2 concisely presents the methodology and shows that such an approach is feasible and desirable. Section 3 presents the results of our experiments, which were conducted on the AWS (Amazon Web Services) Educate platform [2]. Section 4 discusses many-to-many relationships and we shall conclude and state the future research goals of this project in Section 5.

## II. METHODOLOGY

In a nutshell, the proposed design strategy is summarized as follows:

- Capture an application of interest with a conceptual-model hypergraph [8], or some other means that can be translated to a conceptual-model hypergraph.
- Identify the access patterns of the most commonly executed queries and operations in the conceptual-model hypergraph.
- Simplify the conceptual-model hypergraph by identifying object sets that are in one-to-one correspondence with each other.
- Generate a set of hierarchical schemas from the access patterns based on Nested Normal Form [9].

- Load the data of the application to Amazon DynamoDB in accordance to the generated hierarchical schemas.

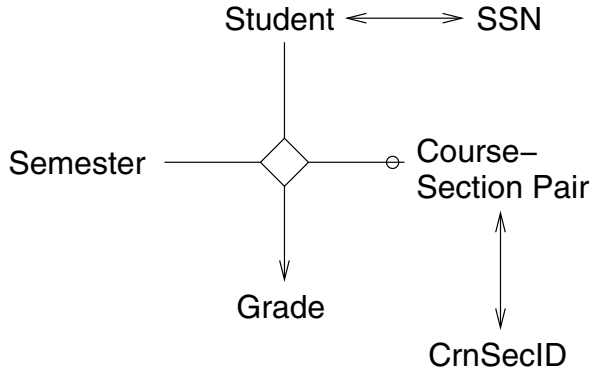


Fig. 1: A sample conceptual-model hypergraph.

#### A. Conceptual-Model Hypergraphs

A highly abstracted conceptual-model hypergraph for a simple reporting system of a small college is shown in Fig. 1. To show the essential features of the proposed design strategy, many details of the hypergraph are omitted. For example, the name, address, picture, and other personal items of a student are nowhere to be found in the hypergraph. Here, we identify the salient features of the hypergraph, although similar explanations can also be found in [8].

The hypergraph of Fig. 1 has six sets of objects, which are called Student, SSN (Social Security Number), Semester, Course-Section Pair, CrnSecID, and Grade. The sets Student, SSN, Semester, and Grade are all self-explanatory. The set Course-Section Pair contains the pairs of every course and all the sections of that course. As an example, an element of that set might be IS301-01, which represents Section 1 of the course IS301: Introduction of Information Systems. We use a six characters string in this paper to represent a course-section pair and CrnSecID is the set of all such strings.

There are three relationship sets among the sets of objects in Fig. 1. The first relationship set, represented as a line, is between the sets Student and SSN. It is called a relationship set because it is a set of relationships. As an example, relationships, or elements, of that set might be (student<sub>1</sub>, 111223333) and (student<sub>2</sub>, 222334444), representing student<sub>1</sub> is related to the SSN 111223333 and student<sub>2</sub> to 222334444. The line between Student and SSN has arrow heads at both ends, which represents a one-to-one correspondence between the sets Student and SSN. In our example, student<sub>1</sub> is thus paired with 111223333 and so is student<sub>2</sub> with 222334444. The second relationship set is between the sets Course-Section Pair and CrnSecID, which is also represented as a line. Because both ends of that line have arrow heads, each course-section pair is paired with exactly one CrnSecID. The third relationship set is among the sets Student, Semester, Grade, and Course-Section Pair, represented as a diamond. There is a

single arrow head of this relationship set, pointing at the object set Grade. It represents the functional dependency (FD) Student, Course-Section Pair, Semester  $\rightarrow$  Grade, which means that for each tuple of a Student element, a Course-Section Pair element and a Semester element, there can only be one Grade element. The circle close to the set Course-Section Pair means that a particular course-section pair might or might not relate to the elements of the other three sets. In other words, there are some existing course-section pairs are not used in the relationship set.

#### B. Access Patterns of the most commonly executed Queries and Operations

Suppose that the current semester is Summer 2020 (2020S). At the end of 2020S, every instructor will enter a grade for every student in his/her classes. The reporting system thus needs to present a class list to each instructor and the instructor will enter a grade for every SSN on the class list. Hence, there are two access patterns, which are shortened as AP1 and AP2, and their involved object sets of Fig. 1 are outlined in Fig. 2.

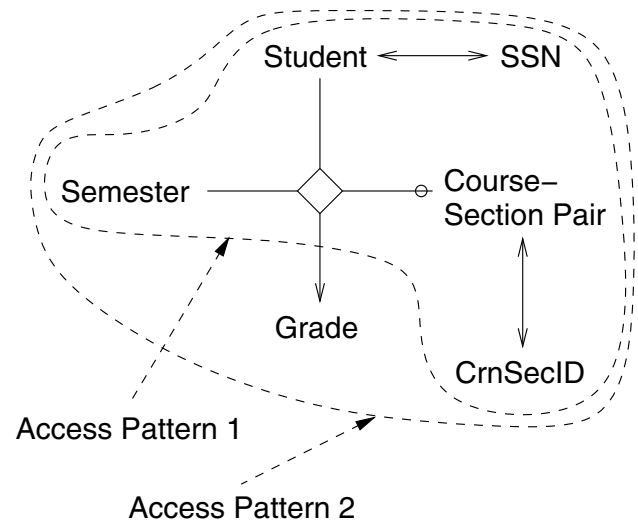


Fig. 2: Access patterns of two commonly executed query and update operation at the end of 2020S.

AP1: The query that generates a class list of 2020S will first select the semester 2020S and a particular CrnSecID. Then a list of SSNs is generated. Thus, this query involves all of the object sets in Fig. 1 except Grade.

AP2: The update operation is to record the grade earned by a student for any class of 2020S for which the student registered. Thus, this operation involves every object set of the relationship set in Fig. 1.

#### C. Simplification of Conceptual-model Hypergraphs

Because of the one-to-one correspondence between the object sets Student and SSN, each student can be replaced by his/her SSN. The same is also true for the object sets

Course-Section Pair and CrnSecID. We thus obtain the simplified hypergraph in Fig. 3.

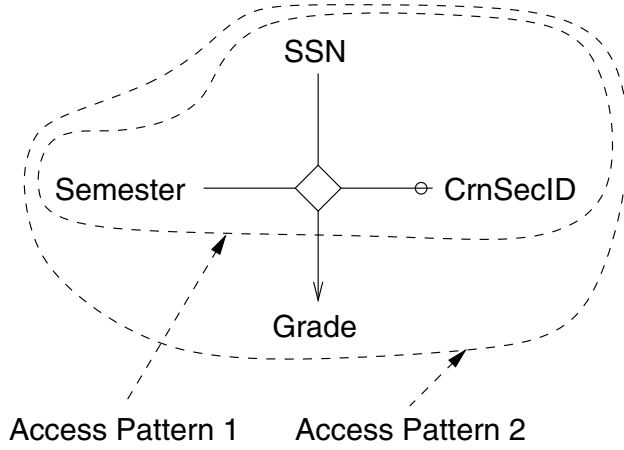


Fig. 3: A simplified conceptual-model hypergraph.

#### D. Generation of Hierarchical Schemas

The algorithms in [8] can generate hierarchical schemas from any conceptual-model hypergraph with any number of relationship sets and each relationship set can contain any number of object sets. The hypergraph in Fig. 3 only has one single relationship set that contains four object sets. First, note that the single relationship set of Fig. 3 is in BCNF [6], meaning that it does not have data redundancy with respect to the FD  $SSN, CrnSecID, Semester \rightarrow Grade$ . Based on Nested Normal Form, the object sets of the relationship set can be organized in any single path hierarchical schema without any unwanted redundant data. Two possible Nested Normal Form hierarchical schemas are shown in Fig. 4.

The left hierarchical schema is derived from Access Pattern 1, denoting the fact that for each pair of CrnSecID and Semester, there is a set of SSNs. Note that the object set Grade is left out because Grade is not needed in the query that generates the class list of any class of 2020S. The right hierarchical schema is derived from Access Pattern 2. It means that for each pair of SSN and Semester, there is a list of pairs of the form CrnSecID and Grade.

### III. RESULTS

#### A. Simulation Background

The experiments were conducted on AWS Educate, a free platform for cloud learning [2]. A Cloud9 environment, which is a cloud IDE (Integrated Development Environment) for writing, running, and debugging code, was first created. We selected the default settings for the environment whose EC2 instance type is t2.micro that has 1 virtual CPU, 1 GB of memory, and a Linux OS. (Amazon Elastic Compute Cloud (EC2) is a web service that provides secure, resizable compute capacity in the cloud.) A real-world production environment will

clearly have a more substantial virtual computing environment.

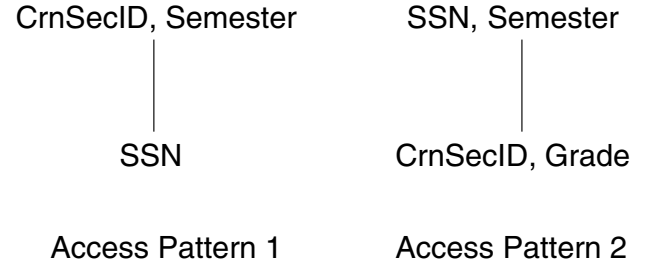


Fig. 4: Two hierarchical schemas generated from the access patterns in Fig. 3.

After which, a Python program generated two DynamoDB tables, called IEEM2020-CrnSec and IEEM2020-Students where IEEM2020-CrnSec is to hold the data of the left hierarchical schema in Fig. 4 and IEEM2020-Students for the right one. The read and write capacity of each table were set to 500 units, which costs about \$290 monthly for each table. The partition key of IEEM2020-CrnSec is set to Semester and the sort key to CrnSecID. This is appropriate because the records of IEEM2020-CrnSec ought to be partitioned according to Semester, and within each semester the records are further sorted according to their CrnSecIDs. Each Semester and CrnSecID pair in IEEM2020-CrnSec has a list of SSNs where a DynamoDB list is similar to a Python list. For the table IEEM2020-Students, the partition key is SSN and the sort key is Semester. This is also appropriate because students are the focus of IEEM2020-Students and thus SSN partitions the table. The records of a student are further sorted according to Semester. Each SSN and Semester pair in IEEM2020-Students is associated with a list of maps where a DynamoDB map is similar to a Python dictionary.

Note that the root nodes of the two hierarchical schemas each consists of two object sets. Although such choices are dictated in part by the semantics of the application, they are chosen also because primary keys of DynamoDB tables can only contain two or less attributes. Note that this is a severe limitation because the primary key of the relationship set in Fig. 3 is in fact SSN, Semester, CrnSecID and many real-world primary key may consist of more than two attributes.

#### B. Simulation Data

We've randomly generated 1,000 through 10,000, in the increment of 1,000, active students to simulate the workload of the reporting system at the end of 2020S. Only active students have been simulated because only active students have registered for classes in 2020S and only they will receive a grade for their classes of 2020S. Each active student has enrolled in classes for a number  $x$  of semesters where the integer  $x$  follows an exponential distribution with the scale set to 6. Hence, most active students have enrolled in less than 5 semesters and at most 12 semesters. Each active student has registered a number  $x$  of courses in a

semester where the integer  $x$  follows a normal distribution with the mean and standard derivation respectively set to 4 and 1. All of these data were written out to JSON (JavaScript Object Notation) files, preparing to be loaded into the DynamoDB tables.

```
{
  "SSN": 100327004,
  "Semester": "2020S",
  "CrnSecList": [{"432754": " ", "567017": " ", "716933": " ", "290347": " "}]
},
{
  "SSN": 100327004,
  "Semester": "2020W",
  "CrnSecList": [{"317239": "D", "312013": "A", "772832": "A", "294905": "D"}]
},
...
}
```

Fig. 5: Some randomly generated active student records.

Fig. 5 shows a part of the JSON file that contains some randomly generated student records about an active student whose SSN is 100327004, and the semesters for which he/she enrolled (2020S and 2020W), and the grade he/she earned for each CrnSecID of which he/she registered. Note that the grades for 2020S are all blank.

```
{
  "Semester": "2020S",
  "CrnSec": "116447",
  "StdList": [143267942, 167456489, 191522130, 203634078, 218949345, 228395488,
    262223274, 299359064, 340291307, 383893398, 402528408, 429940466, 491781834,
    503895266, 524376955, 555747721, 588971187, 622489021, 632925410, 683107756,
    707819456, 736073179, 746724915, 753864030, 759208116, 763603644, 764238408,
    775280136, 783255531, 808439142, 814791444, 827636210, 865237592, 921601262,
    942888972, 969574148, 972276135]
},
{
  "Semester": "2020S",
  "CrnSec": "143078",
  "StdList": [111183936, 143267942, 156225049, 157319776, 163586751, 224053635,
    233214797, 234530365, 262458151, 295119560, 321794227, 329578153, 331832654,
    393320569, 406218919, 459185184, 503871987, 509132188, 523055023, 547079471,
    587846611, 589394822, 614437773, 650503546, 659577543, 665339504, 665548465,
    738868279, 741704917, 775280136, 816330964, 823199550, 877078492, 897820459,
    923783761, 944155639, 977837091]
},
...
}
```

Fig. 6: Some randomly generated class lists.

Fig. 6 shows a part of the JSON file that contains the class lists of certain pairs of CrnSecID and 2020S. Each class list is essentially a list of SSNs

### C. Simulation Results.

The query that retrieves every class list of 2020S and the update operation that assigns a grade to every class of each active student of 2020S are written as parts of a Python program, shown in Fig. 7. The code segment was repeated 5 times, and the elapsed time was printed out at the end of the loop.

```
import timeit

code_to_test = """
import random
import boto3
from boto3.dynamodb.conditions import Key

dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
tableC = dynamodb.Table('IEEM2020-CrnSec')
tableS = dynamodb.Table('IEEM2020-Students')

respC = tableC.query(KeyConditionExpression=Key('Semester').eq('2020S'))

for item in respC['Items']:
    for sid in item['StdList']:
        respS = tableS.update_item(
            Key={'SSN': sid, 'Semester': '2020S'},
            ExpressionAttributeNames={
                "#crnsec": item['CrnSec'],
            },
            ExpressionAttributeValues={
                ":grade": random.choice(['A','B','C','D','E']),
            },
            UpdateExpression="set CrnSecList.#crnsec = :grade",
        )

"""

elapsed_time = timeit.timeit(code_to_test, number=5)
print(elapsed_time)
```

Fig. 7: The Python program that retrieves every class list of 2020S and updates the grade of every class of every active student.

Since the SSNs of every class list are randomly generated, every student has an equal probability of being updated. Therefore, there are no localities of updates. Table 1 shows the average time the Python program took to perform an update and Fig. 8 shows the information graphically. Note that DynamoDB took less than 0.0085 second to perform an update for no more than 10,000 students.

TABLE I  
AVERAGE TIME OF AN UPDATE

| Num of students | Num of course section pairs | Num of iterations | Num of updates | Elapsed time | Average time of each update |
|-----------------|-----------------------------|-------------------|----------------|--------------|-----------------------------|
| 1000            | 100                         | 5                 | 3975           | 140.821      | 0.00709                     |
| 2000            | 200                         | 5                 | 7924           | 298.602      | 0.00754                     |
| 3000            | 300                         | 5                 | 12027          | 448.486      | 0.00746                     |
| 4000            | 400                         | 5                 | 16109          | 547.187      | 0.00679                     |
| 5000            | 500                         | 5                 | 20015          | 746.519      | 0.00746                     |
| 6000            | 600                         | 5                 | 24031          | 885.607      | 0.00737                     |
| 7000            | 700                         | 5                 | 28020          | 1116.735     | 0.00797                     |
| 8000            | 800                         | 5                 | 31924          | 1293.276     | 0.00810                     |
| 9000            | 900                         | 5                 | 35851          | 1353.335     | 0.00755                     |
| 10000           | 1000                        | 5                 | 40207          | 1470.509     | 0.00731                     |

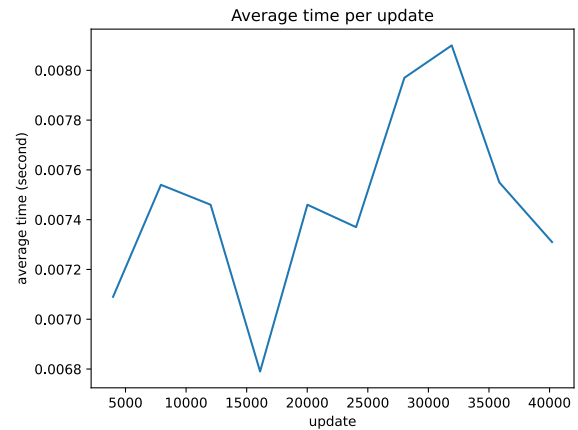


Fig. 8: Plotting average time of an update against number of updates.

To perform a stress test on the update operations on the database, we've randomly generated a long text string to store with each SSN and Semester pair in the table IEEM2020-Students, where the length of every text string follows a normal distribution with the mean and standard derivation set to 4,000 and 100 bytes respectively. We've also inserted a long text string for every pair of Semester and CrnSecID in the table IEEM2020-CrnSec. The purpose of these long text strings in both tables are to ensure that there are great physical distances among the updates. The results of the stress test are shown in Table 2. Note that even though long text strings have been inserted in both tables, the average time for each update in IEEM2020-Students is not much different than the case that has no long inserted text strings.

TABLE II  
AVERAGE TIME OF AN UPDATE WITH LONG TEXT STRINGS  
INSERTED.

| Num of students | Num of course section pairs | Num of iterations | Num of updates | Elapsed time | Average time of each update |
|-----------------|-----------------------------|-------------------|----------------|--------------|-----------------------------|
| 1000            | 100                         | 5                 | 3992           | 140.662      | 0.00705                     |
| 2000            | 200                         | 5                 | 7976           | 300.682      | 0.00754                     |
| 3000            | 300                         | 5                 | 11997          | 470.264      | 0.00784                     |
| 4000            | 400                         | 5                 | 15956          | 567.652      | 0.00712                     |
| 5000            | 500                         | 5                 | 19935          | 732.344      | 0.00735                     |

#### IV. DISCUSSION

Astute readers might discover that the relationship between a SSN, a Semester and a CrnSecID is being stored in two distinct places, once in the table IEEM2020-CrnSec and once in IEEM2020-Students, although the same relationship is being stored with different focuses in the two tables. The Developer Guide for Amazon DynamoDB in fact recommends using adjacency list to model many-to-many relationships, where we can see that a many-to-many relationship is also being stored in two distinct places in the adjacency list. A basic tenet of the theory of relational databases is that redundant data always lead to high update cost. However, many practitioners of NoSQL databases oftentimes champion denormalization in order to improve performance. This, of course, deserves more study.

#### V. CONCLUSION

In this paper, we have presented evidence for the feasibility of a DynamoDB schema design strategy, which is based on Nested Normal Form and past experiences on XML databases. Such a design strategy begins with a conceptual-model hypergraph, which is in a way similar to UML domain class diagrams. It then generates hierarchical schemas in Nested Normal Form that are guaranteed to be free of unwanted redundant data. The generated hierarchical schemas are then mapped to DynamoDB tables for implementation.

We have performed experiments to further show the usefulness of the proposed design strategy. We have randomly generated 1,000 through 10,000 active students, in the increment of 1,000, and each active student has enrolled in a number of semesters that follows an exponential distribution. In addition, each active student has registered for a number of courses that follows a normal distribution in each semester. The results show that on average each update has taken less than 0.0085 second. We've also subjected the database to a stress test where a long text string was inserted into a student record and a course-section record. The purpose of which is to ensure that updates are not done in close localities. It turns out that the average update time is about the same with or without the inserted long text strings.

Much more work remains to be done. Currently we construct a hierarchical schema for each assess pattern, which may lead to redundancy. However, we shall study

the update cost for making such a choice. As more results are being developed, a future journal paper shall report the finer details of the proposed schema design strategy.

#### REFERENCES

- [1] "Amazon DynamoDB: Fast and flexible NoSQL database service for any scale," 2020. Accessed on: May 30, 2020. [Online]. Available: <https://aws.amazon.com/dynamodb/>
- [2] "AWS Educate: Your Cloud Journey Starts Here," 2020. Accessed on: May 30, 2020. [Online]. Available: <https://aws.amazon.com/education/awseducate/>
- [3] "BaseX: The XML Framework," 2020. Accessed on: May 30, 2020. [Online]. Available: <http://basex.org/>
- [4] G. Booch, J. Rumbaugh and, I. Jacobson, *The Unified Modeling Language User Guide*. Addison Wesley object technology series, Addison-Wesley, 2005
- [5] "Cassandra: Manage massive amounts of data, fast, without losing sleep," 2020. Accessed on: May 30, 2020. [Online]. Available: <https://cassandra.apache.org/>
- [6] D. Maier, *The Theory of Relational Databases*. Computer Science Press, 1983.
- [7] W.Y. Mok, and D.W. Embley, "Generating Compact Redundancy-Free XML Documents from Conceptual-Model Hypergraphs," *IEEE Trans. Knowl. Data Eng.*, vol. 18, no. 8, pp. 1082–1096, 2006.
- [8] W.Y. Mok, J. Fong, and D.W. Embley, "Generating the fewest redundancy-free scheme trees from acyclic conceptual-model hypergraphs in polynomial time," *Inf. Syst.*, vol. 41, pp. 20–44, 2014.
- [9] W.Y. Mok, Y.K. Ng, and D.W. Embley, "A Normal Form for Precisely Characterizing Redundancy in Nested Relations," *ACM Trans. Database Syst.*, vol. 21, no. 1, pp. 77–106, 1996.
- [10] "MongoDB: The database for modern applications," 2020. Accessed on: May 30, 2020. [Online]. Available: <https://www.mongodb.com/>
- [11] "NoSQL Databases." 2020. Accessed on: May 30, 2020. [Online]. Available: <https://www.trustradius.com/nosql-databases>
- [12] "Redis: Experience the fastest NoSQL database in the cloud." 2020. Accessed on: May 30, 2020. [Online]. Available: <https://redislabs.com/>
- [13] "Visual Paradigm: Stay Competitive and Responsive to Change Faster & Better in the Digital World," 2020. Accessed on: May 30, 2020. [Online]. Available: <https://www.visual-paradigm.com/>